Resource benchmark

Measured on the reference load profile. Numbers below are produced by `scripts/run_benchmark.py`; regenerate them rather than editing by hand.

Check environment

- OS: Linux-6.8.0-1055-gke-x86_64-with-glibc2.41
- Kernel: 6.8.0-1055-gke
- CPU: x86_64 (4 logical cores)
- RAM: 15990 MiB
- Python: 3.13.15
- dpkt: 1.9.8

The reference environment in the specification is Linux x86-64, 4 vCPU, 8 GB RAM. The numbers below are from the machine shown above.

Input

- Input capture: 95.66 MiB
- Packets read: 250000
- TCP sessions: 1000
- Read diagnostics: 0
- Target profile: 250000 packets, 1000 sessions, up to 100 MiB, responses up to 65536 bytes

Timings

Phase	Seconds
Parse (read_capture)	7.29
Sessions (build_sessions)	0.70
Reassembly (reassemble)	1.84
Export (export_capture)	2.27
Total	12.10

Streaming mode

`process_streaming` reads the capture in blocks and writes each session as soon as it can no longer receive packets, so closed sessions leave memory. The output is identical to the regular mode; only the resource profile differs. The streaming peak is measured in its own process so the two do not share a high-water mark.

Metric	Regular	Streaming
Total seconds	12.10	11.48
Peak RSS (MiB)	350	224
Output JSON (MiB)	122.44	122.44

The streaming run flushed 1000 sessions.

Streaming used less memory.

Memory and output

- Peak process RSS: 350 MiB
- Output JSON: 122.44 MiB

Conclusions and limitations

- Peak memory is 350 MiB, well under the 4 GiB threshold.
- Total time is 12.10 s, well under the 5 minute threshold.

Observations:

- The parser dominates the time. It builds one Python object per packet, which is the cost of keeping the header fields available for provenance.
- The normalized JSON (122.44 MiB) is larger than the input (95.66 MiB) because stream bytes are stored base64 encoded and every observed range carries a provenance record.
- Memory scales with the capture size: all packets are held in memory at once. At this profile that is affordable; for a capture several times larger the same approach would need to stream or use memory mapping.

If a larger capture must be supported, the first step is to stop holding every packet object at once: parse and reassemble session by session, and write provenance ranges to disk as they are produced. That trades peak memory for a second pass over the file.

Size sweep: regular vs streaming

The same capture is normalized twice, regular then streaming, so the two modes are compared on identical input. Each size is a distinct generated capture with the same session and message shape; `packets` scales with the size. Memory is the peak resident set of the process.

Input	Packets	Sessions	Regulars	Regular peak MiB	Streamings	Streaming peak MiB	Output MiB
9.57 MiB	25000	100	1.10	58	1.06	54	12.20
47.84 MiB	125000	500	6.10	186	5.71	167	61.12
90.88 MiB	237500	950	11.46	336	10.86	293	116.32

Phase breakdown by size

Seconds per phase of the regular pipeline. Reassembly includes session identification.

Input	Parse	Sessions	Reassembly	Export	Total
9.57 MiB	0.69	0.067	0.16	0.19	1.10
47.84 MiB	3.73	0.357	0.96	1.05	6.10
90.88 MiB	6.83	0.646	1.84	2.13	11.46

Defects capture

`tests/corpus/corpus_capture_defects.pcapng` is small but carries a retransmission, an out-of-order segment, a gap, and a conflicting overlap. It is measured to confirm the diagnostics do not degrade the throughput at this size.

Metric	Regular	Streaming
Input	1.7 KiB	same
Packets	18	18
Sessions	1	1
Seconds	0.021	0.022
Peak RSS (MiB)	29	29

Python allocation (tracemalloc, 10 MiB)

Peak memory tracked by `tracemalloc` during a regular run over the 10 MiB capture. This counts Python allocations only, not the memory the interpreter or libraries hold outside the allocator, so it is below the process RSS.

Metric	Value
Input	9.57 MiB
Packets	25000
tracemalloc peak	31 MiB
Process RSS	93 MiB